Journal of Engineering Design

ISSN: 0954-4828 (Print) 1466-1837 (Online) Journal homepage: www.tandfonline.com/journals/cjen20

Ergonomic modelling and optimization of the keyboard arrangement with an ant colony algorithm

Marc Oliver Wagner, Bernard Yannou, Steffen Kehl, Dominique Feillet & Jan Eggers

To cite this article: Marc Oliver Wagner, Bernard Yannou, Steffen Kehl, Dominique Feillet & Jan Eggers (2003) Ergonomic modelling and optimization of the keyboard arrangement with an ant colony algorithm, Journal of Engineering Design, 14:2, 187-208, DOI: 10.1080/0954482031000091509

To link to this article: https://doi.org/10.1080/0954482031000091509

Published online: 02 Aug 2010.

Submit your article to this journal

Article views: 200

View related articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=cjen20

J. ENG. DESIGN, VOL. 14, NO. 2, JUNE 2003, 187–208

Ergonomic modelling and optimization of the keyboard arrangement with an ant colony algorithm

MARC OLIVER WAGNER†, BERNARD YANNOU†*, STEFFEN KEHL†, DOMINIQUE FEILLET† and JAN EGGERS†

The arrangement of letters on a keyboard greatly influences the user's comfort and typing speed. With repetitive muscular injuries emerging and the amount of text to be treated increasing, the need for a better arrangement has come forth. To solve the problem, a non-trivial abstract representation of a physical keyboard layout and of type-writing rules is introduced. Having justified the choice of heuristic ergonomic criteria, a mathematical function for the ergonomic evaluation criteria is developed. Based on the ant colony optimization algorithm, an optimization algorithm is designed and applied to the problem described. The computational results representing the quality of the keyboard arrangements obtained for several languages are finally discussed and new solutions for optimization of the keyboard are presented.

1. Introduction

A computer user or typist is easily surprised when first looking at the seemingly arbitrary arrangement of letters on a standard computer keyboard. Neither does the arrangement have any alphanumeric logical order, nor is it optimized according to a high hit rate or ergonomic comfort. In fact, it is more the result of the historical development of the typewriter market and, notably, the achieved dominance of the Remington II typewriter that imposed the arrangement. Introduced by the Sholes brothers in 1873, the keyboards using this arrangement have become known as Sholes– or *QWERTY*–keyboards. Initially designed in an English-speaking context, they were slightly re-arranged and thus adapted to other languages such as French and German. This gave rise to the *AZERTY* and *QWERTZ* versions. These counter-optimized keyboards were justified for some decades, during which mechanical problems remained in typewriters. With electronic typewriters and personal computers emerging it was no longer the case but, despite some serious work to get optimized keyboard arrangements, the *QWERTY* layout remained the standard.

This fact might be partially explained by pointing out that all new suggestions, notably those issued by August Dvorak in the US and Claude Marsan in France, have so far been manually designed. This implies a high probability that they are

Revision received January 2003.

† Laboratoire Génie Industriel, Ecole Centrale Paris, Grande Voie des Vignes, 92295 Châtenay-Malabry Cédex, France.

* To whom correspondence should be addressed. e-mail: yannou@pl.ecp.fr

Journal of Engineering Design
ISSN 0954-4828 print/ISSN 1466-1387 online © 2003 Taylor & Francis Ltd
http://www.tandf.co.uk/journals
DOI: 10.1080/0954482031000091509

suboptimal and therefore risk lacking the necessary impact to induce a change of standards. Indeed, a repeated change of standards is excluded.

A second obstacle for the introduction of a new standard is the lack of reliable evaluation criteria to justify a change of standards. Ergonomic research in the field of computer keyboards has so far not been able to supply a satisfying set of rules for the ergonomic evaluation of a keyboard arrangement. The only set one can hope for therefore consists of a set of heuristic rules that derive from common sense and are verified by experimental studies.

The primary contribution of this paper is to propose an overall design process for the ancient keyboard arrangement problem (KAP), which is based on the one hand on a strong modelling of a physical keyboard layout and of typewriting complexity, and on the other hand on an original optimization approach. The second contribution is the modelling of the problem. Up to now, the KAP has never been addressed with a very realistic modelling approach. Our model is not trivial, since it includes realistic features for typing such as upper-case/lower-case letters, multi-key symbols (as shift keys), or typewriting rules (space key can be hit by both hands, etc.), which had never been considered before. Furthermore, the evaluation criteria qualitatively rely on realistic ergonomic rules set up 25 years ago by Marsan (1976, 1979, 1987) after heavy ground experiments. They have been mathematically combined into an evaluation function (Kehl and Wagner 2000) and quantitatively assessed and weighted with the help of ergonomists. This way, we hope that the evaluation of the quality of a keyboard arrangement really reflects the reality of typewriting. A third contribution concerns the original implementation of ant colony optimization (ACO) algorithms to the KAP. This choice is motivated by the mathematical structure of the problem. The originality of the KAP relies on its complex and time-expensive objective function. A keyboard arrangement is not assessed with a simple analytic formula, but its evaluation requires one to simulate the writing of a (long) text. In our approach, an iteration of the solution algorithm (the construction of a keyboard arrangement by an ant) is fast, and only an assessment of the complete keyboard is needed per ant (typically 30 ants) and per algorithm cycle (typically 2000 cycles), resulting in a low number of complete keyboard assessments. In comparison, for a classic local search approach (e.g. taboo search), the assessment of a change in the arrangement (e.g. the permutation of two letters or two rows) would have implied a complete or at least partial—but lengthy—re-calculation of the objective function. Actually, ant algorithms seem to be perfectly devised for the KAP. Finally, the present paper presents and justifies new out-ranked English, French and German ergonomic keyboard arrangement layouts.

The paper is organized as follows: in section 2 the different types of keyboard arrangement problems are presented and an historical view over the relevant types is given. Section 3 then defines the basic vocabulary, introduces abstract physical keyboard layout and typewriting models, and suggests some restrictions on the general model with respect to the optimization phase. Section 4 is dedicated to the presentation and justification of the heuristic ergonomic rules. In section 5, the procedure developed to obtain the necessary statistical data is described. The algorithm that serves for the optimization procedure is presented in section 6, and section 7 summarizes the computational results by comparing the newly found arrangements with traditional ones. Finally, section 8 concludes the work and gives a perspective for future research.

2. The keyboard arrangement problem

2.1. Different types of arrangement problems

Research concerning the arrangement of letters on a keyboard has so far been conducted primarily in two directions, and thus has led to two different definitions of a keyboard arrangement problem. The first problem considers the situation where the number of letters (for keyboard) exceeds the number of positions available. This leads to an inherent ambiguity, since several letters are assigned to the same key. For instance, these kinds of problem are found for touch-tone telephones where the alphabet consists of 26 letters and the keyboard consists of only eight keys, and where two or three letters are assigned to each key (see Oommen *et al.* (1992) or the assignment of alphabets based on the Chinese language to European keyboards in Glover (1986)). The primary objective is to determine the sets of characters that may be assigned to one key while causing just a minimum of ambiguity for a given set of words. To cite an example from Oommen *et al.* (1992), the sequence '269' on a touch-tone telephone matches the word 'boy' as well as the word 'box'. The number of such words, which have the same representation on the keyboard and which can therefore not be exactly determined from the input, has to be minimized. It is worthwhile to point out that the problem definition does not consider the question of the association of the sets of character to the keys once these sets of character have been determined. The set $\{a, b, c\}$, for instance, may thus be assigned to any key without changing the result of the optimization problem.

The second type of problem is based on the idea that every character of the alphabet is produced by a unique sequence of keys. Thus, there is no ambiguity in the word representation. The focus of the problem lies in the absolute and relative positioning of the keys on the keyboard in order to optimize a criterion such as typing speed (Norman and Fisher 1982), rapid learning of the typing system (Norman and Fisher 1982) or ergonomics (Kehl and Wagner 2000).

This paper considers a problem of the second type, which will subsequently be called the KAP. The chosen optimization criteria related to the typing ergonomics are defined in section 4.

2.2. History of the KAP

When the first commercial typewriter was invented in 1867, the notion of ergonomics was practically unknown. The primary criteria for evaluating the difference between the suggested models therefore became the possible hit rate. Due to the mechanical set-up of the machines working with little hammers that punched a ribbon in order to produce a letter on the paper, the limiting factor for the hit rate was the inertia of the hammers. When two keys were struck successively, there was the danger of them sticking together and therefore demanding a significant pause to re-set the machine. This restriction led to a counter-optimization of the arrangement. Letters were set up to guarantee a maximum distance of the position of consecutive letters and often used letters were positioned at the extremities of the keyboard.

With the invention of the electronic typewriter, the mechanical restrictions became obsolete. In the 1930s, the American researcher Dr August Dvorak started his work on a new ergonomic arrangement of the letters, allowing for a higher typing speed. Due to the lack of computers, the research took more than a decade before the first results were obtained.

Innovative ideas were not restricted to the US. Claude Marsan, a French researcher who had collaborated with Dvorak in the US, slightly modified Dvorak's assumptions on the ergonomics of keyboards and undertook a research for the French language.

However, although the American National Standards Institute approved the Dvorak layout in 1982, and some companies sell their Dvorak layout keyboards on the web, neither Dvorak nor Marsan succeeded in imposing their arrangements as a market standard.

3. Modelling of the keyboard

A keyboard basically serves to enter a string of characters into a memory by striking a sequence of keys. An abstract definition of a keyboard can therefore be the following: a set of typable characters and the respective sequences of keys that enable their construction as well as a geometrical set up of the keys. A general keyboard model has to take these aspects into account to allow the description of a large family of keyboards to which the existing standard keyboards inevitably have to belong. In the following section, some important terms to describe the model of the keyboard are introduced.

3.1. Important definitions

Character A character is the smallest visible unit of a text. It is important to note that 'r' and 'R' are different characters and that ' ' and the end of a paragraph are also characters.

Symbol A symbol is the denomination for a character. In most cases, the denomination and the character can be identical; however, in some cases, a different denomination is useful (e.g. *space* for ' ').

Real key A real key is a physical key on the keyboard described by its position.

Special key A special key is a set of real keys that have the same function and thus the same denomination (e.g. two keys for *shift*). It is defined by its denomination, the set of reals keys it represents and the rule that determines which of the keys is used in a given situation. Special keys need a real key to be struck afterwards in order to produce a character.

Macro A macro is a sequence of real and special keys that activates the character of a given symbol. For all symbols other than space, it is unique.

3.2. Definition of a keyboard model

To define a keyboard model, four objects have to be defined: the geometric layout, the sets of characters and symbols, as well as the list of macros.

Concerning the geometric layout, the fact that keyboards might have a different number of rows, columns and keys has to be taken into account. Most commercial keyboards have a layout that corresponds to a rectangle divided into rows and columns. Those conventional keyboards present offset rows—in a presently unjustified ergonomic way—which is due to ancient mechanical considerations—alternation of rods between keys and hammers. Each column is generally assigned to a finger, and

a so-called 'home row' is defined where the fingers lay in a relaxed position. Therefore, an abstract keyboard layout may be summarized in a key matrix, each key being defined by its geometric position:

- a hand (left, right);
- a column (0–7 for the left hand and 0–8 for the right hand, 0 standing for the thumbs); and
- a row (0–5, 0 standing for the top row and 3 for the rest row).

This given number of columns and rows allows the representation of most of the presently available keyboards, notably the *QWERTY* keyboard and its German (*QWERTZ*) and French (*AZERTY*) equivalents, as well as the French *Marsan* optimized keyboard (Marsan 1976, 1979, 1987). The *AZERTY* keyboard is represented in the abstract model in figure 1.

Subsequently, the optimization is conducted in order to fit on an ergonomic physical keyboard layout, named ECP, with some important patented characteristics (Yannou and Hossenlopp 2000). Among those are two palm-rests with integrated keys that can be struck by the thumbs and the forefingers, respectively. The keys for the forefingers are put on an extra row (number 5) while the keys activated by the thumbs are on an extra column (number 0). The authors have already worked on fitting the

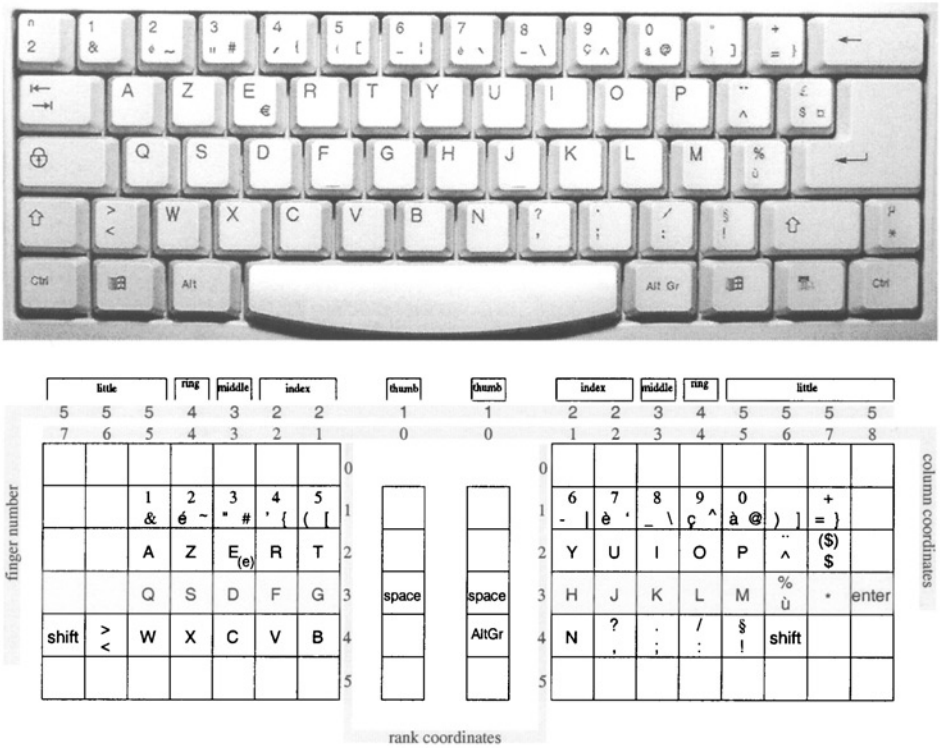

Figure 1. Transcription of the *AZERTY* keyboard.

Marsan optimized keyboard layout into the ECP layout (see figure 2), leading to the so-called '*initial French ECP keyboard*'.

As a first alphabet, the complete set of characters available on a standard keyboard is used. The alphabet is subject to simplifications so as to reduce the complexity of the problem (see section 3.3).

For a given keyboard, the list of macros is easy to establish (i.e. sequences of keystrokes leading to the symbols). But to set up a new keyboard, the sequence of keystrokes leading to a symbol has to be designed in a more or less complex manner. Without any supplementary condition, there is an infinite number of possible layout solutions in terms of macro list solutions since any sequence of keys may be a candidate for a given symbol. Since a finite solution space is needed to apply one of the metaheuristics designed to solve combinatorial optimization problems, certain restrictions have to be imposed.

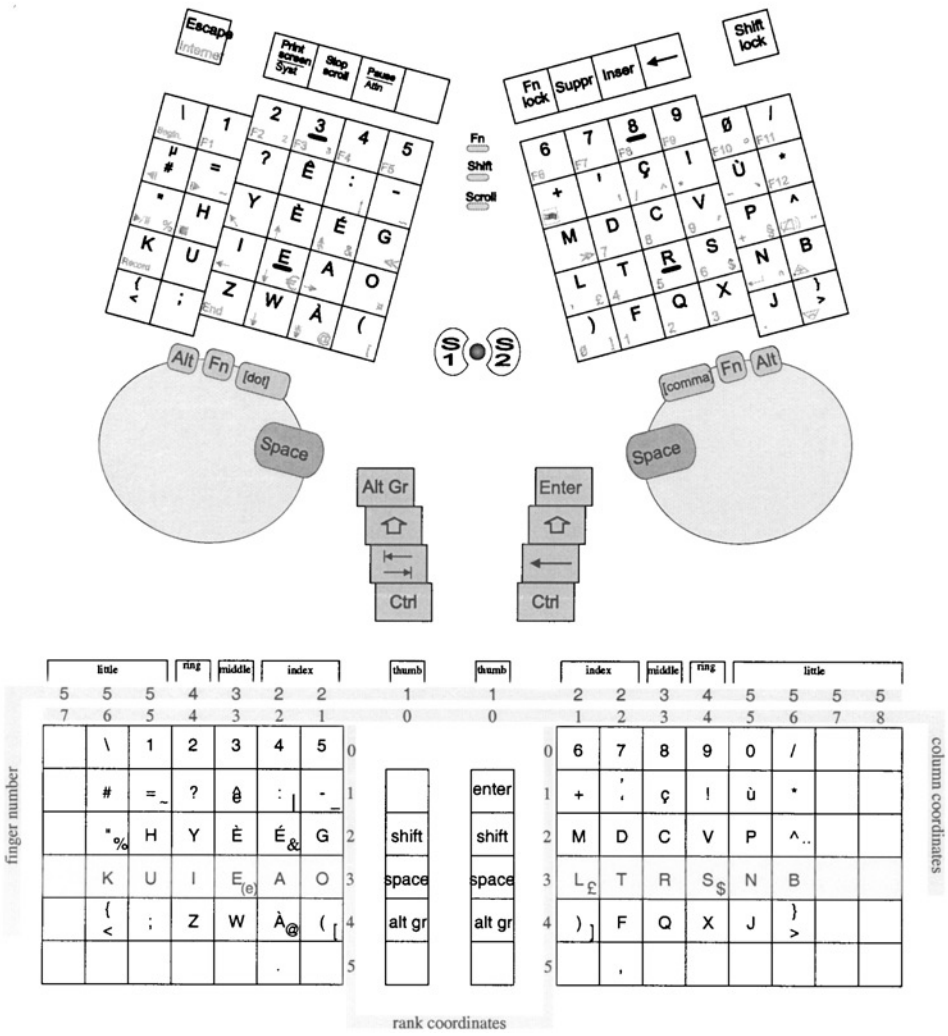

Figure 2. Transcription of the initial French ECP keyboard.

3.3. Reduction of the problem space

To reduce the size of the problem space, certain assumptions have been translated into rules. First of all, the considered alphabet is reduced. Certain characters are not considered because of their rareness in usual texts such as '°' '/', *Alt* and *Ctrl*; others such as *backspace* because of the impossibility to determine their frequency from a text source. It is worthwhile to notice that the rate of error and thus the frequency of *backspace* strongly depends on the arrangement of the keys.

In addition, several keys are imposed to be used in combination to reduce the amount of possible macros. Indeed, one has adopted the most widespread solution for implementing macros as:

- a sequence of one or 'more *modifier key*(*s*)' as *shift* or *AltGr* followed by one or 'more *conventional key*(*s*)';
- or a unique '*conventional*' *key.*

An additional simplification is adopted in fixing the position of modifier keys. A useful definition for a future optimization can then be as follows:

Combination character set A combination character set is a set of symbols whose macro definitions refer to a one conventional key.

For example, the capital letter 'E' is composed by striking *shift* and a key that is used to produce the character 'e'. Symbols 'E' and 'e' therefore form a combination character set. The complete combination character set table may be found in Eggers *et al.* (2001b).

As a consequence, an optimization problem may be considered along with the two following notions:

Elementary assignment An elementary assignment is a single association of a combination character set to a real key.

Keyboard solution A keyboard solution is a complete set of elementary assignments, where every combination character set is associated to a real key.

4. Ergonomic criteria

4.1. Our design process and choices

A keyboard design should comply with four main objectives:

- Allow one to type a text without tiredness.
- Maximize typing speed.
- Reduce the number of typing errors.
- Allow a rapid mastery of the touch-typing method.

As already pointed out, these objectives cannot be easily translated into mathematical criteria that could serve for an evaluation of a given keyboard. Indeed, a primary difficulty can be encountered in the different types of exhaustion and their potential pathological nature. Short-term exhaustion reduces typing speed but does not present a danger to the writer's health, whereas long-term exhaustion might lead to disorders such as cumulative trauma disorders. A second uncertainty lies in the localization of exhaustion. Since typing requires movements of fingers, arms and shoulders and since

those movements are produced by a combination of actions of muscles, joints and tendons, it is difficult to quantify exhaustion.

Because of the lack of exact ergonomic criteria, we decided to start with a set of sound ergonomic rules that ergonomic keyboard layouts should comply with. Marsan (1976, 1987) established such a set of rules that he precisely described in his patents. These rules are solidly grounded on experiments that he carried out on two series of more than 30 typists. Both groups of typists, initially beginners, were equally trained on, respectively, classical *AZERTY* keyboards and *Marsan* keyboards. During several intensive typing sessions, different measurement protocols of tiredness were applied. These experiments resulted into a set of ordered ergonomic rules often composed themselves of ordered preferences (as order of preference for key rows, relative strength of fingers, etc.).

Lately, our work has been twofold. First, combining the initial Marsan laws into a set of six aggregated criteria expressed by mathematical formulas. This work is presented hereunder, but more details can be found in Kehl and Wagner (2000). This re-expression has also been inspired by other works such as (Norman and Fisher 1982) and (Burkard and Offermann 1977). Second, providing values to quantify amounts of preferences and the weights between criteria. These quantifications have been carried out with the help of two ergonomists, sometimes with the help of a Pairwise Comparison approach (i.e. a decision-making method) to result in a weighting of a set of criteria to make the best use of intuition in a group (see Limayem and Yannou (2001) for the determination of the weights of the six criteria).

4.2. The evaluation function

Using a given keyboard layout, any text may be broken up into a sequence of real keys that have to be struck in order to type the text. This sequence can be seen as a sequence of single real keys—called monographs—or a sequence of two real keys—the digraphs. Ergonomic criteria are based upon the distinction between rules that concern the absolute position of a given real key and the relative position of two consecutive real keys.

A first step toward the valuation of a keyboard layout therefore consists of establishing the statistical appearance of monograph and digraphs according to the keyboard used. Subsequently, f_{m_i} and f_{d_i} present the statistical appearance of the monograph m_i and the digraph d_i, respectively. This statistical data serves for the evaluation of the following ergonomic criteria.

4.2.1. Accessibility and load. The combination character sets should be distributed on the keys in a way that the total load is shared adequately by all fingers. The fact that some keys are less accessible than others also has to be taken into account. So as to evaluate a grade, an optimal hit load has to be defined for each key. In a first step, a value is assigned to each hand representing a possible difference between their performances and endurances. Since the two ergonomists we consulted did not agree on giving the right hand an advantage over the left hand, the ratio is chosen to be 50% for each hand. Then, each column receives a coefficient that takes into account the relative agility and endurance of the fingers. Here, the relative strength of the fingers is known to be greatest for the index, then the middle finger, followed by the little and the ring finger and, finally, by the thumb. Their values come from orders of magnitude given by an ergonomist researcher (Richardson 2000). Then, each row is

given a value representing the relative accessibility of that row preferring the home row over all other rows. The optimal load distribution $f_{m_i}^{opt}$ is then calculated by multiplying the three corresponding values for each key. Figure 3 gives a graphical representation of the chosen ideal distribution. It presents the nine columns assigned to the different fingers, starting from the left with one column for the thumb, two columns for the forefinger, a column for the middle and ring finger, respectively, and four columns for the little finger. The third row represents the home row and the height of the bars, the ideal hit load normalized to have a maximum of 10 000. Additionally, table 1 provides the initial values chosen for the rows and the columns.

The grade evaluates the variance of the load distribution on a keyboard from the ideal distribution and is given by the following equation:

$$v_1 = \sum_{m_i \in \Xi_1^m} (f_{m_i} - f_{m_i}^{opt})^2 \tag{1}$$

where Ξ_1^m is the set of all monographs.

4.2.2. Key number. To minimize the total hit load, the number of hits necessary to compose a given text has to be minimized. The indicator v_2 is therefore calculated by dividing the number of characters in a text by the number of keystrokes necessary to produce the text. While it is relatively important for the most general definition, this grade does not vary between the different solutions of the KAP. This is due to the introduction of the combination character sets, which fixes the number of keys necessary for each character. It should nevertheless be mentioned that the keyboard that allows the highest productivity in terms of words per minute, the so-called chord keyboard, requires significantly more hits than a standard keyboard (see Alden *et al.* 1972).

4.2.3. Hand alternation. The fastest and most comfortable typing is assured when subsequent keys are not hit by the same hand. To quantify the hand alternation indicator, we sum the frequency of the digraphs that are typed by using one hand only. In terms of mathematical formulation, the indicator is calculated as follows:

$$v_3 = \sum_{d_i \in \Xi_3^d} f_{d_i} \tag{2}$$

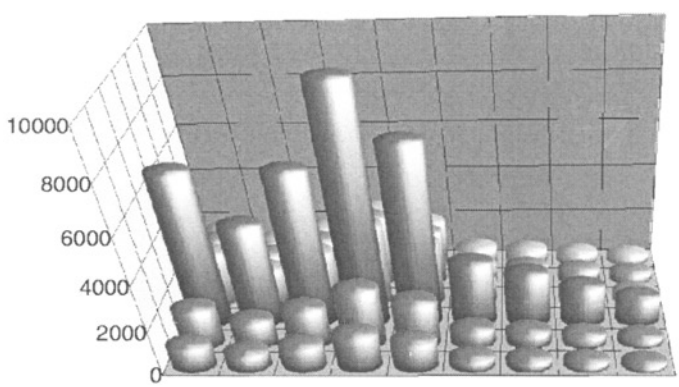

Figure 3. Ideal load distribution for the keys activated by the right hand.

Number	Row (%)	Column (%)
0	10.87	15.38
1	13.04	10.26
2	15.22	15.38
3	43.48	23.08
4	10.87	17.95
5	6.52	6.41
6		5.13
7		3.85
8		2.56

Table 1. Ideal load distribution.

where Ξ_3^d is the set of all digraphs that are typed using one hand only.

4.2.4. Consecutive usage of the same finger. The preceding alternation rule is also true for the fingers. Two consecutive keys should not be hit by the same finger. The indicator is calculated by summing the frequencies of the corresponding digraphs and multiplying each of them with a distance coefficient. The greater the distance, the more inconvenience for a consecutive usage. The relevant set Ξ_4^d is therefore the set of digraphs that are typed using the same finger of one hand. The derived equation is:

$$v_4 = \sum_{d_i \in \Xi_4^d} f_{d_i} dist(d_i) \tag{3}$$

The chosen distance function is the Manhattan distance given by:

$$dist(d_i) = |c_2 - c_1| + |r_2 - r_1| \tag{4}$$

where c_2 and c_1 are the respective columns of the two keys that establish the digraph, and r_2 and r_1 are the corresponding rows.

4.2.5. Avoid big steps. When one hand is used for two consecutive hits, great distances that require awkward hand postures should be avoided. This is the case for keys whose vertical distance is greater than or equal to one. The relevant set Ξ_5^d is therefore the set of digraphs that are typed using the same hand, but not the same finger, and the vertical distance between the two keys is greater than or equal to one row. A weight coefficient depending on the two fingers used is assigned to each digraph. The coefficient increases with the following pairs of fingers, 1 representing the thumb and 5 the little finger (2–3, 2–5, 3–5, 2–4, 3–4, 4–5). Finally, the indicator is given by:

$$v_5 = \sum_{d_i \in \Xi_5^d} \kappa(d_i) f_{d_i} \tag{5}$$

where $\kappa(d_i)$ is the weight coefficient. The chosen heuristic values for $\kappa(d_i) = \kappa(i, j)$ are represented in table 2, i and j representing the first and second finger, respectively.

	Thumb	Forefinger	Middle finger	Ring finger	Little finger
Thumb	0	0	0	0	0
Forefinger	0	0	5	8	6
Middle finger	0	5	0	9	7
Ring finger	0	8	9	0	10
Little finger	0	6	7	10	0

Table 2. Big step coefficients.

4.2.6. Hit direction. For digraphs typed by one hand only, the preferred hit direction is from the little finger towards the thumb. This is the natural finger movement for most people, which may easily be verified by tapping on the table according to the two possible directions. Ξ_6^d is therefore the set of all digraphs that are produced using one hand only and whose hit direction is not the preferred one. The indicator is given by:

$$v_6 = \sum_{d_i \in \Xi_6^d} f_{d_i} \tag{6}$$

4.2.7. Global grade. To define a global grade, let us consider the six indicators v_j $(1 \leq j \leq 6)$. Since these indicators have different ranges and units, and since they have different relative importances, they are divided by the respective indicators of a reference keyboard $v_{j,\mathrm{ref}}$. Once the indicators are turned dimensionless they can be multiplied by a relative weight coefficient γ_j and summed. This relative global grade, which will be our optimization criterion, expresses the fact that one will try to improve keyboards from existing keyboards (in practice, they will be the English *QWERTY*, the French *AZERTY* and the German *QWERTZ* keyboards).

The values of weights γ_j, represented in table 3, have been obtained after a working session with two ergonomists and with the help of a decision-making method for weighting in a group. This use of a pairwise comparison method on the KAP is detailed as a case study in Limayem and Yannou (2001).

This leads to the final formula:

$$V = \sum_{j=1}^{6} \frac{v_j}{v_{j,\mathrm{ref}}} \gamma_j \tag{7}$$

Rule	γ
Load and accessibility	0.45
Key number	0.5
Hand alternation	1.0
Consecutive usage of the same finger	0.8
Avoid big steps	0.7
Hit direction	0.6

Table 3. Weight coefficients γ.

that defines the evaluation of a keyboard for the KAP. The lower the grade, the better the keyboard arrangement.

5. Pre-processing of statistical data

To assess the subsequent ergonomic criteria, two lists must inevitably be established for a given language: the list of monographs of keys, and the list of digraphs of keys. These two lists are generated by parsing a given text with the macro definitions and then transcribing the sequence of characters into a sequence of real keys. This process of data transformations is now explained.

Three major newspapers/magazines served as the sources for the text to be parsed: *USA Today* for English, *Le Monde* for French and *Der Spiegel* for German (see Kehl and Wagner (2000) for the size and exact references of these texts).

First of all, each character has to be assigned to a symbol in order to give it a name tag. The most natural way is to assign to each character the character itself as a symbol and to name functional keys according to their function (e.g. *shift* and *retun*).

To give an example of the procedure, the parsing of the sequence 'le Ce', which is part of the string 'Ecole Centrale'—the name of the institution where the presented research has been conducted—is demonstrated here.

As a second source of information, the definition of the virtual and real macros has to be known.

5.1. Virtual macro list

The virtual macro list serves as an intermediate stage between the physical model of the keyboard and the real macro list. Indeed, at this stage, special keys involved in the virtual macro list are not instantiated into real keys (see next section). In table 4 an excerpt of the virtual macro list, corresponding to the string 'le Ce' is given for the *AZERTY* keyboard—keys are described as (hand, column, row).

5.2. Decision rules

For all special keys like *shift*, *AltGr*, and so on, a decision rule has to be defined that determines which of the real keys that are part of the special key have to be struck. On the physical model, the special key has to be pressed while the real key that follows is struck. Since releasing a key does not require a significant amount of concentration or time, only the striking is taken into account. According to standard typing rules, both hands should be used to produce the character. The decision rule can therefore be stated to be: the hand that strikes the following key is determined, the special key being struck by the other hand.

Symbol	Key 1	Key 2	Symbol	Key 1	Key 2
c	(0,3,4)		C	Sh	(0,3,4)
e	(0,3,2)		E	Sh	(0,3,2)
l	(1,4,3)		L	Sh	(1,4,3)
lSp	(0,3,0)		rSp	(1,3,0)	
lSh	(0,7,4)		rSh	(1,6,4)	

Table 4. Partial list of virtual macros.

Symbol	Key 1	Key 2	Symbol	Key 1	Key 2
c	(0,3,4)		C	(1,6,4)	(0,3,4)
e	(0,3,2)		E	(1,6,4)	(0,3,2)
l	(1,4,3)		L	(0,7,4)	(1,4,3)
lSp	(0,3,0)		rSp	(1,3,0)	
lSh	(0,7,4)		rSh	(1,6,4)	

Table 5. Partial list of real macros.

The only special macro that is not unique is the one for *space* since the key might be struck by any hand. To determine the hand that strikes the key, the following rule is used: the hand that strikes the previous key is determined and *space* is struck by the other hand.

These two rules being defined, the virtual macro list can be transcribed into a real one.

5.3. Real macro list

When the symbols are replaced by real keys according to the corresponding rules explained in section 5.2, the virtual macro list is transformed into the real one. In the given example, *space* has to be replaced by *rspace*, since the preceding key is struck by the left hand and then substituted by the position of the physical key. This leads to the list presented in table 5. All the necessary elements being defined, the two major lists can finally be established.

5.4. List of monographs of keys

To obtain the sequence of keys, the sequence of characters has to run through the following substitutions. Each character is first replaced by a symbol that represents the character. Then, the symbol is substituted by the corresponding special macro, which is unique for each symbol. Finally, the special macro is transformed into a real macro by taking the decision rules into account. The whole process is illustrated in figure 4. As the final result, the sequence of real keys is obtained and the frequency of each key that occurs in the text is calculated by dividing its number of occurrences by the total

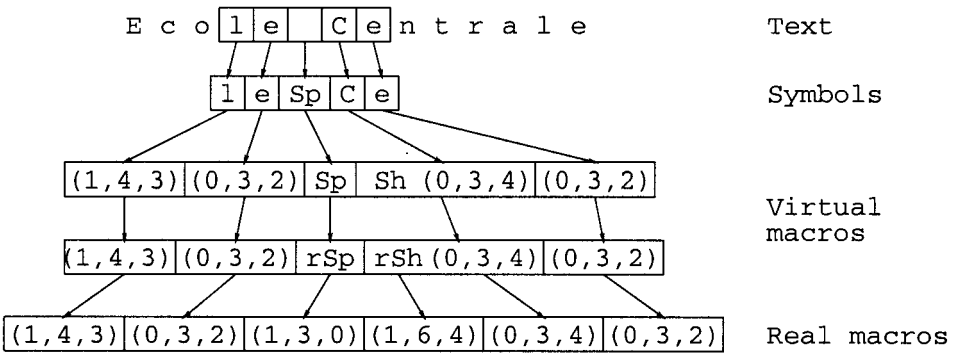

Figure 4. Translation of a source text into a series of keystrokes.

Key	Frequency	Key	Frequency
(1,4,3)	0,166	(0,3,2)	0,333
(1,3,0)	0,166	(1,6,4)	0,166
(0,3,4)	0,166		

Table 6. List of monographs.

number of occurrences of monograph (see Table 6). The list of the digraphs of keys is obtained in the same way.

One can notice that our apparently complicated model allows a discoupling between the frequencies of characters found in a text and those of physical key strokes. This is not the case for most of the studies on the keyboard arrangement previously conducted. Moreover, it allows to take the difference between upper-case and lower-case letters into account.

6. Implementation of the optimization algorithm

6.1. Reasons for using an ACO algorithm for the KAP

The KAP is a discrete, combinatorial optimization problem. Ant colony algorithms were first introduced for solving combinatorial problems in Dorigo *et al.* (1991), and have subsequently been applied to several other optimization problems. So far, they have proven to be a powerful and easily adaptable tool that yields excellent results over highly combinatorial problems.

The evaluation function of the KAP is neither linear nor convex. As such, the natural solution strategy may be found in metaheuristic optimization algorithms such as taboo search, genetic algorithms, simulated annealing or ACO algorithms (Pham and Karaboga 2000).

The major characteristic for choosing an ACO-based algorithm for the KAP is the complexity and difficulty of evaluation of the objective function (global grade) for a given entire keyboard layout. Compared with more classical problems such as the traveling salesman problem (TSP) or the quadratic assignment problem (QAP), the time necessary to calculate the global grade is excessive. An efficient algorithm should therefore not demand the whole evaluation of many solutions.

Indeed, most optimization algorithms such as simulated annealing and genetic algorithm and most local search procedures use the global grade and an abstract memory as the only means to direct the search. In contrast, ACO algorithms additionally take advantage of local information in order to gradually make global solutions emerge through an associated memory: the pheromone matrix, an autocatalytic process. This local information and the associated memory are both easily accessible and relatively cheap in terms of computational time, and very few global cost-expensive keyboard evaluations are needed. The successful application of ACO algorithms to several different optimization problems such as the TSP (Dorigo and Gambardella 1997; Stützle and Dorigo 1999) and the QAP finally led to the conclusion that their potential to solve the KAP might exceed the potential of other approaches. The excellent results we have obtained proved we were right. Of course, a comparative study with other metaheuristics would remain to be carried out.

6.2. The ant metaphor for the KAP

The fundamental idea of ACO algorithms is based on the behaviour of natural ants who succeed in finding the shortest paths from their nest to food sources by communicating via a collective memory that consists of pheromone trails. Due to its weak global perception of its environment, an ant moves essentially at random when no pheromone is present. However, it tends to follow a path with a high pheromone level when many ants move in a common area, which leads to an auto-catalytic process. Finally, the ant does not choose its direction exclusively on the level of pheromone, but also takes the proximity of the nest and of the food source, respectively, into account. This allows the discovery of new and potentially shorter paths.

Applied to the keyboard assignment problem, the algorithm can be defined as follows: a colony consists of N ants. Each ant is given a string of characters, which is extracted from the reference text that serves as the source for statistical data, and an empty keyboard (i.e. a keyboard without a macro list). It then arranges the combination letter sets that correspond to the characters of the string on its own keyboard in a random manner, but taking successful placements in the past into account and respecting the ergonomic rules established in section 4 as much as possible. Once every ant has assigned every combination character sets to a key, a cycle is finished and the keyboards are evaluated. The ants leave pheromone on the elementary assignments that are part of its solution. These pheromone placements are summarized in a pheromone matrix (τ_{ij}), where i is a combination character set and j the associated key. The amount of pheromone that each ant leaves is proportional to the quality of their solution. Then the pheromone is partially evaporated, the keyboards are cleared and the ants re-start the search process with new character strings until a pre-defined stopping criterion is met.

6.3. Principles of our algorithm

Our algorithm is a combination of several versions suggested by the ACO research community. As a basic framework, the ACO algorithm presented in Dorigo *et al.* (1999) was used because it displays the most promising features. By emphasizing the importance of the best solution found, the basic ideas of AS_{Elite} (Dorigo *et al.* 1991, 1996, 1999) are added by assigning more pheromone to the ant that found the best solution during one cycle. MAX–MIN ant systems (Dorigo *et al.* 1999, Stützle and Hoos 1998) contribute to the final algorithm, because a minimum and a maximum are defined for the pheromone level. Finally, AS_{Rank} (Bullnheimer *et al.* 1997) introduces the choice of the ants that are allowed to update the pheromone according to the respective ranking of their solutions.

Since there are some interesting starting points for the search in the form of existing good quality keyboard arrangements such as the *Dvorak* or the *Marsan* keyboards, it is possible to modify the standard ACO algorithm that demands a uniform initial distribution of pheromone and to consider a non-uniform initialization of the pheromone matrix. Such a distribution—first suggested by Corne *et al.* (1999)—translates an *a priori* knowledge of the subspace in which the optimal solution may be found.

The obvious drawback of this initialization is the fact that an optimal solution outside the determined subspace is found with a smaller probability than in the case of a uniform initial pheromone distribution.

A second modification is due to the fact that it is impossible to guarantee that all the characters are present in the text string used by a given ant to establish its keyboard. Indeed, some rarely used combination character sets are usually not attributed

during a cycle. This possibly results in an incomplete keyboard at the end of the construction phase and prohibits a valid evaluation of the quality of the keyboard. Due to their minor importance, the remaining sets are randomly assigned to the remaining keys once the end of the character string in reached.

6.4. Description of our algorithm

From the principles mentioned, the following algorithm has been deduced. This algorithm consists of two parts. The first is an initialization phase that creates the necessary data structure and sets all parameters to their initial value. The second part is the iterative phase of the algorithm that is repeated until a stopping criterion is satisfied. This criterion is defined to be a certain amount of cycles.

6.4.1. Initialization. During the initialization phase, the following tasks are carried out:

1. An ant colony of N agents is created.
2. A text source is defined.
3. An empty character string of fixed length l is assigned to each ant.
4. An empty keyboard is assigned to each ant.
5. A pheromone matrix is created and its elements are set to values that take an *a priori* knowledge of the solution in the form of high-quality keyboard arrangements into account.
6. The algorithm parameters (see definitions later) are initialized.

6.4.2. Iteration. The second part of the algorithm is repeated until a certain number of iterations has been completed. The subsequent steps are:

1. The keyboards assigned to the ants are emptied.
2. The strings assigned to the ants are initialized using the text source.
3. Each ant reads its string and places the combination character sets on its keyboard until the end of the string is reached. This procedure is described in detail in section 6.4.3.
4. The remaining combination character sets are randomly assigned to the remaining keys.
5. Each ant evaluates the solution it has found.
6. The pheromone matrix τ evaporates with a coefficient ρ_{all}:

$$\tau \leftarrow \rho_{all} \cdot \tau \tag{8}$$

7. A number p of the best solutions are selected and pheromone is updated according to their rank: The kth best ant, with $1 \leq k \leq p$, deposits a quantity $\Delta\tau \cdot \mathbf{q}(k)$ of pheromone on each elementary assignment that it has chosen, where $\Delta\tau$ represents a quantity of pheromone and the vector $\mathbf{q}$ is a measure of importance of the best ants.
8. The pheromone matrix is modified to fit the allowed range $[\tau_{\min}, \tau_{\max}]$.

6.4.3. Character placement. The subsequent steps allow an ant to choose the most adequate key to place the combination character set i under consideration:

1. For a given character to process, the ant checks if its corresponding combination character set has already been placed on the keyboard. If so, the

ant skips the following steps and continues with the next character of the string.

2. For all free keys j, the ant calculates a probability according to the random proportional decision rule:

$$p_{ij} = \frac{[\tau_{ij}]^{\alpha} \cdot [\eta_{ij}]^{\beta}}{\sum_{k \in \Omega} [\tau_{ik}]^{\alpha} \cdot [\eta_{ik}]^{\beta}} \tag{9}$$

   where Ω is the set of not yet occupied keys and η_{ij} is the global grade calculated according to equation 7 when only the present and the previously assigned keys are considered. It is worthwhile to note that the grade η_{ij} only requires a few calculations. α and β are two coefficients describing the relative importance of experience and locally good elementary assignment. The ant uses the calculated probabilities to establish a random variable, according to which it determines a key among those that are still free.
3. The ant places the combination character set on the chosen key.
4. The ant evaporates pheromone on the elementary assignment found according to the following equation:

$$\tau_{ij} \leftarrow \rho \cdot \tau_{ij} \tag{10}$$

   ρ being the coefficient of evaporation.

More details concerning ACO approaches of the literature and our specific ACO adaptation for the keyboard design are given in Eggers *et al.* (2001a).

7. Computational results

The experimental work consisted of two major phases. First, an efficient parameter setting was determined. Then, the algorithm was run several times using this setting with and without initializing the pheromone matrix.

7.1. Parameters setting

It is known that the convergence depends strongly on the parameters affecting the computational formula (see Dorigo *et al.* 1991). In the case of the presented algorithm, N, α, β, ρ, ρ_{all}, $\Delta\tau$, $\tau_{\min}$, $\tau_{\max}$, p, l and $\mathbf{q}$ had to be determined. As a starting point for a heuristic determination of the parameters, those used in Dorigo *et al.* (1991) were chosen. Observing the development of the pheromone matrix, the parameter settings were adjusted experimentally. The best parameters found are summarized in table 7.

Since the objective is to find a nearly optimal keyboard arrangement, the algorithm is not meant to be for repetitive use. Once a quasi-optimal solution is found, the

N	30	α	3.0	β	3.0
ρ	0.98	ρ_{all}	0.95	$\Delta\tau$	0.2
$\tau_{\min}$	0.09	$\tau_{\max}$	1.0	p	4
l	6000	$\mathbf{q}$	[1, 0.5, 0.2, 0.1]		

Table 7. Choice of optimization parameters.

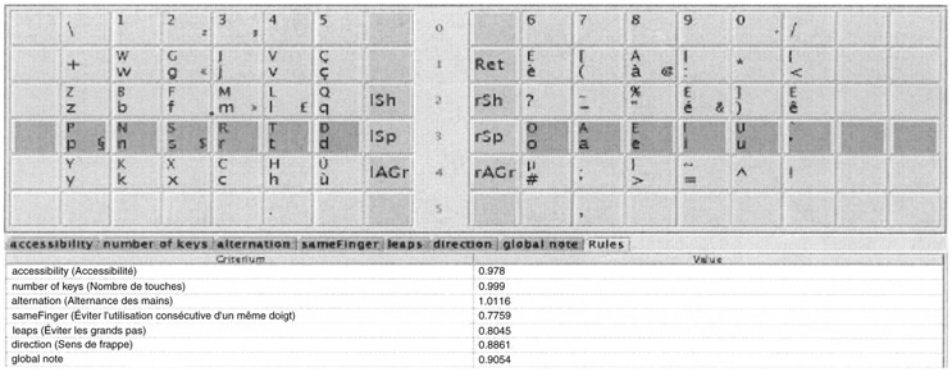

Figure 5. The algorithm output for the French language, assessed in comparison with the initial ECP arrangement.

algorithm is not used any more, the only apparent re-use being the application of the algorithm to other languages. So as not to waste too much time, a statistical examination of different parameter settings was left out. However, slight changes in some parameters frequently led to a significant lowering of the quality of the solution.

7.2. Optimized keyboard arrangements

As a direct output of the ACO algorithm, the best arrangement found for French is presented in figure 5. It is 51% better than the French *AZERTY* standard. It required about 2000 iterations, which corresponds to about 14 hours on a 400 MHz PC. The best arrangement found for English is given in figure 6 and is 41% better than the English *QWERTY* standard. The best arrangement found for German is given in figure 7 and is also 41% better than the German *QWERTZ* standard.

These arrangements require six rows and therefore cannot be mapped on the standard keyboard layout; they require an ergonomic ECP layout (described in Yannou and Hossenlopp (2000)) as presented in figure 2.

7.3. Quality of designs

The best keyboards produced by the algorithm always have the same main characteristics. All the vowels are struck by the same hand and no frequently used consonants are found on that side of the keyboard. The relative position of the vowels and the most

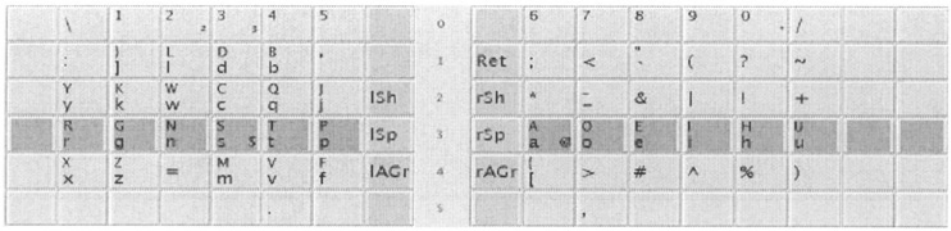

Figure 6. The algorithm output for the English language.

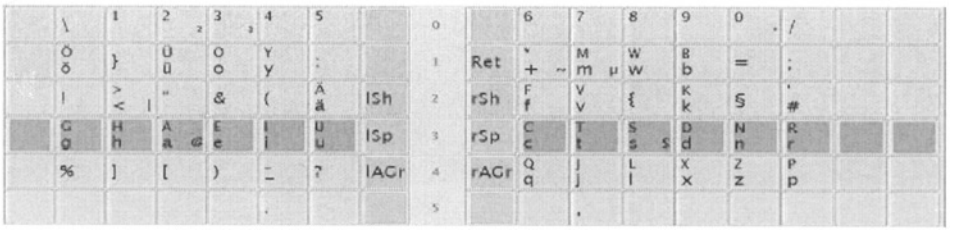

Figure 7. The algorithm output for the German language.

frequently used consonants are identical for most of the runs. This partitioning property is similar to those of *Dvorak* (in the English context) and *Marsan* (in the French context) manually optimized keyboards, which is intellectually satisfactory. Furthermore, the keyboards found out-rank these well-known *Dvorak* and *Marsan* manually optimized keyboards. The qualities of the keyboards have been summarized in table 8.

For French, the optimization algorithm has improved the *Marsan* keyboard by 19%. The authors previously worked on a manual adaptation of the *Marsan* arrangement (Marsan 1976, 1979, 1987) over the ECP physical layout, leading to an '*initial ECP arrangement*' keyboard given in figure 2. This keyboard was already 11.5% better than the *Marsan* keyboard but the optimization algorithm yielded a keyboard design (see figure 5) that is 9.4% better than this initial ECP arrangement. Finally, a manual additional modification was performed to reinforce certain symmetries and proximities that were not considered in the ergonomic criteria. It leads to our definitive optimized proposition, named the final ECP arrangement, represented in figure 8. Reinforcing certain symmetries and proximities facilitates memorizing the new positions of the letters on the keyboard. For example, brackets are positioned symmetrically and certain characters are grouped together, as 'e', 'é', 'è' and 'ê'. These modifications, that the algorithm did not allow to affect, further slightly increase the quality of the keyboard arrangement by 1.9%. This ultimate arrangement has finally been suggested as a standard layout for future French keyboards.

These satisfactory design quality improvements might partially be explained by the lack of adequate computational machines at the time when the majority of works in the area of the re-distribution of the letter were realized. Therefore, most previous works in the domain led to simplifications in the models that introduced several minor disadvantages. The fact that *space* and the difference between ordinary and capital letters was neglected is just one example. However, certain aspects found during optimization agree with the simplifications that prior researchers assumed to hold. To cite but one example, Marsan established (1976, 1979, 1987) a rule that demanded all vowels be put on the same side of the keyboard. Although not demanding such a rule, all algorithm outputs had in common that this rule was fulfilled.

For the English language, the optimization algorithm has improved by only 1.9% the *Dvorak* keyboard (see figure 6). This apparently disappointing result proves that the English keyboard was better studied by researchers. But our program tends to show that the *Dvorak* keyboard arrangement is close to an optimal keyboard design.

	AZERTY	*Marsan*	Initial ECP arrangement	Algorithm output	Final ECP arrangement
Load and accessibility	1	0.489	0.491	0.479	0.478
Hand alternation	1	0.687	0.665	0.673	0.657
Consecutive usage of the same finger	1	0.503	0.338	0.263	0.238
Avoid big steps	1	0.315	0.276	0.222	0.224
Hit direction	1	0.454	0.421	0.373	0.373
Global grade	1	0.568	0.518	0.487	0.480
Improvement vs *AZERTY*	0%	43.2%	48.2%	51.3%	52.0%
Improvement vs *Marsan*		0%	11.5%	17.9%	19.3%
Improvement vs initial ECP			0%	9.4%	11.3%

Table 8. Comparisons between the French keyboards: standard *AZERTY* manually optimized *Marsan*, initial ECP arrangement, ant colony algorithm output, and final ECP arrangement.

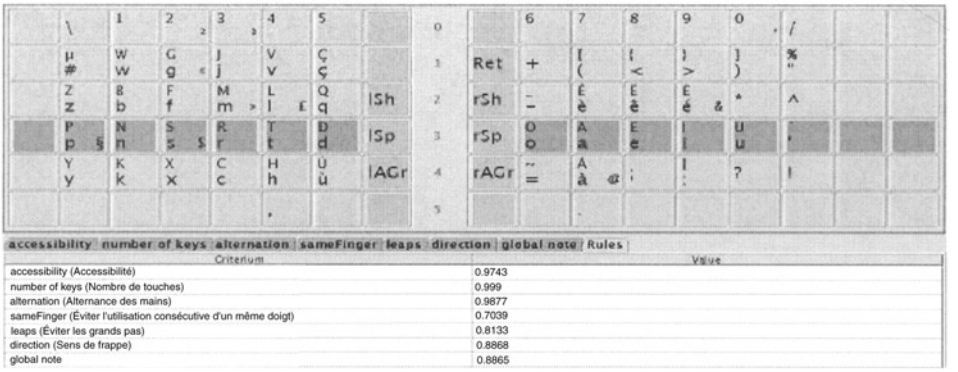

Figure 8. The final ECP arrangement for the French language, assessed in comparison with the initial ECP arrangement.

8. Conclusions

In this paper, the ergonomic optimization of the distribution of the characters over an ergonomic keyboard layout—described in Yannou and Hossenlopp (2000) and represented in figure 2—has been realized for French, English and German. A flexible keyboard model has been introduced and coupled with a statistical analysis of the languages concerned. Second, six ergonomic criteria have been mathematically defined. Third, an ACO algorithm has been proposed that is based on several different ant optimization approaches found in the literature. The results obtained show that the algorithm is effective in finding very good solutions. In particular, all known manually optimized keyboards such as the *Dvorak* keyboard and the *Marsan* keyboard were outranked. This gain in quality might be explained by pointing out that previous research primarily focused on typing speed or on easy memorization of the arrangement. The new results concern a comprehensive approach of the ergonomic quality that the mentioned criteria are just one part of. All the described modelling and optimization process can be applied to any other language as soon as statistical data are established from a representative text.

As the reality of type-writing cannot be completely taken into account in a mathematical function, we tried to qualitatively assess these promising results. Some typists accustomed to a *Marsan* layout in their professional life validated the apparent quality and convenience of the new optimized French layout we proposed. Some quantitative validations remain to be done.

References

ALDEN, D. G., DANIELS, R. W. and KANARICK, A. F., 1972, Keyboard design and operation: a review of the major issues, *Human Factors*, **14**, 275–293.

BULLNHEIMER, B., HARTL, R. F. and STRAUß, C., 1997, A new rank based version of the ant system—a computational study, *Technical Report POM-10/97* (Institute of Management Science, University of Vienna).

BURKARD, R. E. and OFFERMANN, J., 1977, Entwurf von Schreibmaschinentastaturen mittels quadratischer Zuordnungsprobleme, *Zeitschrift für Operations Research*, **21**, B121–B132.

CORNE, D., DORIGO, M. and GLOVER, F., 1999, *New Ideas in Optimization* (McGraw-Hill).

DORIGO, M. and GAMBARDELLA, L. M., 1997, Ant colony systems: a cooperative learning approach to the travelling salesman problem, *IEEE Transactions of Evolutionary Computation*, **1**, 53–66.

DORIGO, M., MANIEZZO, V. and COLORNI, A., 1991, Ant system: an autocatalytic optimizing process, *Technical Report 91-016* (Milan: Dipartimento di Elettronica e Informazione, Politechnico di Milano).

DORIGO, M., MANIEZZO, V. and COLORNI, A., 1996, The ant system: optimization by a colony of cooperating agents, *IEEE Transaction on Systems, Man and Cybernetics —Part B*, **26**, 1–13.

DORIGO, M., DI CARO, G. and GAMBARDELLA, L. M., 1999, Ant algorithms for discrete optimization, *Artificial Life*, **5**, 137–172.

EGGERS, J., FEILLET, D., KEHL, S., WAGNER, M. O. and YANNOU, B., 2001a, An ant colony optimization algorithm for the optimization of the keyboard arrangement problem, *Technical Report CER 01-03 A*, Laboratoire Productique Logistique, Ecole Centrale Paris, August.

EGGERS, J., KEHL, S. and WAGNER, M. O., 2001b, Élaboration des algorithmes d'optimation d'un clavier, Applied scientific project, Laboratoire Production et Logistique, École Centrale Paris, August.

GLOVER, D. E., 1986, *Experimentation with an Adaptive Search Strategy for Solving a Keyboard Design/Configuration Problem*, Ph.D. Thesis, University of Iowa.

KEHL, S. and WAGNER, M. O., 2000, Modélisation et optimisation de la répartition des letters sur un clavier d'ordinateur. Applied scientific project, Laboratoire Production et Logistique, École Centrale Paris, July.

LIMAYEM, F. and YANNOU, B., 2001, Le tri croisé de monté carlo: une boite á outils pour l'aide á la décision coopérative. *Revue Internationale de CFAO et Informatique Graphique*, Ed. Laroisier-Hermes, **16**, 275–295.

MANIEZZO, V. and CARBONARO, A., 1999, Ant colony optimization: an overview, *Technical Report*, (Italy: Scienze dell'Informazione, University of Bologna).

MARSAN, C., 1976, Perfectionnements aux claviers de machines à écrire et similaires, brevet d'invention no. 76-23323, l'Institut National de la Propriété Industrielle, France, 30 July.

MARSAN, C., 1979, Demande de certificat d'addition no. 79-01065, portant sur le brevet no.76-23323: Perfectionnements aux claviers de machines á écrire et similaires, Institut National de la Propriété Industrielle, France, 17 January.

MARSAN, C., 1987, Claviers alphanumériques ergonomiques pour machines à écrire et similaires, brevet d'invention no. 87-03267, l'Institut National de la Propriété Industrielle, France, 3 March.

NORMAN, D. A. and FISHER, D., 1982, Why alphabetic keyboards are not easy to use: keyboard layout doesn't much matter, *Human Factors*, **24**, 509–519.

OOMMEN, B. J., VALIVETI, R. S. and ZGIERSKI, J. R., 1992, Correction to an adaptive learning solution to the keyboard optimization problem, *IEEE Transactions on Systems, Man and Cybernetics*, **22**, 1233–1243.

PHAM, D. T. and KARABOGA, D., 2000, *Intelligent Optimisation Techniques* (Springer-Verlag).

RICHARDSON, J., 2000, private communication, University of Orsay Sud, France.

STÜTZLE, T. and DORIGO, M., 1999, ACO algorithms for the traveling salesman problem, *Technical Report* (Belgium: IRIDIA, Universit de Bruxelles).

STÜTZLE, T. and HOOS, H. H., 1998, Improving the ant system: a detailed report of the MAX–MIN ant system, *Technical Report* (Darmstadt: Technical University of Darmstadt, Department of Computer Science—Intellects Group).

YANNOU, B. and HOSSENLOPP, P., 2000, Clavier alphanumérique ergonomique. brevet d'invention no 98-04588, numéro de publication 2 777 222, 23 June, Ecole Centrale Paris, Chatenay–Malabry, France.